

인증과 미디어 처리 기초

누구나 부르던 API에 자물쇠를 채우고, 들어오는 파일은 문 앞에서 검사한다

오늘의 한 문장 — 열린 API는 공짜 GPU입니다. 누구인지 확인하고(인증), 무엇을 허락할지 정하고(인가), 오가는 길은 암호화하고(TLS), 비밀은 코드 밖에 두고(시크릿), 모든 출입은 기록합니다(로그). 그 위에서 이미지를 안전하게 받아 추론까지 있습니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

왜 자물쇠인가

인증 없는 추론 API에 벌어지는 일 — 서버 다운·비용 폭탄·모델 도용·추적 불가 — 을 시나리오로 설명하고, **API Key**를 고르는 이유를 말할 수 있다.

02

인증을 코드로

`Header`로 키를 받고 `Depends()`로 엔드포인트에 꽂아, 실패하면 **401**을 돌려주는 구조를 만든다.

03

보안의 큰 그림

인증 vs 인가(401/403), 전송 암호화(TLS), 시크릿 관리(.env), 접근 로그 — 자물쇠 **바깥**의 방어선까지 지도를 그린다.

04

입력을 검사한다

이미지를 `UploadFile`로 받고, 타입·크기·디코딩·리사이즈 **4단계 안전장치**를 거쳐 인증+업로드+추론을 한 API로 엮는다.

지금 이 API, 누가 부를 수 있나?

Day 5의 주택 가격 API는 URL만 알면 누구나 부릅니다. "아직 아무도 모른다"는 안전장치가 아닙니다 — 열린 포트는 자동 스캐너가 몇 시간 안에 찾습니다. 인증이 없으면 네 가지 문제가 따라옵니다.

① **서버가 다운된다.** 요청을 초당 수백 건씩 쏟아부으면 서버가 과부하로 멈춥니다. 추론은 요청마다 CPU·GPU를 쓰는 무거운 작업이라, 정상 사용자까지 같이 막힙니다.

② **비용이 새어 나간다.** 클라우드 GPU를 열어 두면 남의 요청이 내 사용량을 올립니다. 내 GPU가 **공짜 GPU**가 되고, 월말 청구서에 예상 밖 금액이 찍힙니다.

③ **모델이 도용된다.** 공격자가 입력-출력 쌍을 대량으로 모아 같은 동작을 하는 모델을 따라 만듭니다(모델 추출 공격). 몇 달 공들인 모델이 API 호출만으로 복제됩니다.

④ **추적이 안 된다.** 에러가 나도 누가 보냈는지 모릅니다. 원인도 못 찾고, 악성 사용자도 못 막습니다 — 막을 대상을 특정할 수 없기 때문입니다.

네 문제 모두 '누구인지 모른다'에서 시작됩니다. 그래서 첫 번째 방어선은 인증입니다. 인증은 부르는 사람이 누구인지 확인하고, 허용된 사용자만 통과시키는 일입니다.

인증 방식 세 가지 — API Key·JWT·OAuth

복잡도와 보안 수준은 API Key < JWT < OAuth 순서입니다. 이 과정은 가장 단순한 **API Key**를 구현합니다.

이 과정의 선택

API Key

동작이 단순하다

요청 **헤더에 키 한 줄**을 씁니다 — `X-API-Key: ...` 또는 `Authorization: Bearer ...`. 서버는 등록 목록과 대조만 해요. 로그인·세션·쿠키가 없어 오늘 바로 구현합니다.

ML API의 표준

OpenAI·Anthropic·Hugging Face 모두 이 방식입니다.

`client = OpenAI(api_key="sk-...")` 한 줄이면 호출 끝 — 사람이 아니라 **프로그램**이 부르니, 키 하나로 인증과 사용량 추적이 동시에 됩니다.

한계도 분명

키가 유출되면 누구든 그 키로 행세하고, 평문 저장이면 더 위험합니다. 그래서 안전 저장·주기적 교체·폐기 같은 **운영 절차**(§2.10)가 따라붙어요.

더 무거운 방식

JWT · OAuth

JWT — 서명된 토큰

로그인 후 발급되는 토큰에 사용자 ID·만료 시각을 담고 서버가 **서명**합니다. 토큰 자체가 신원을 증명해, 매 요청 DB 조회가 필요 없습니다.

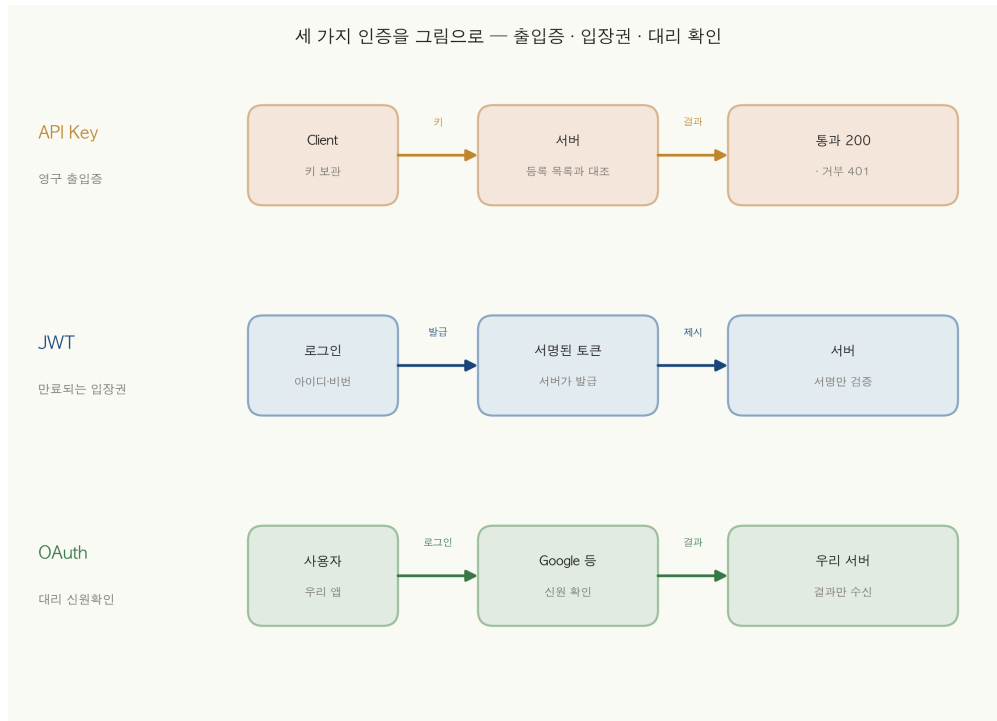
OAuth — 인증 위임

"구글로 로그인"처럼 **외부 서비스에 신원 확인을 맡깁니다**. 비밀번호를 우리 서버가 만질 일이 아예 없어지죠.

언제 필요한가

로그인 세션·만료·세밀한 권한·자동 갱신이 필요할 때입니다. 기본 API Key엔 이게 없어요. 사람이 로그인하는 서비스엔 JWT·OAuth가, 기계가 기계를 부르는 추론 API엔 API Key가 맞습니다.

출입증 · 입장권 · 대리 확인 — 흐름이 다르다



01 API Key = 출입증 한 장

한 번 받은 키를 **매 요청 헤더에 그대로** 신고, 서버는 등록 목록과 대조합니다. 만료가 없어 편하지만, 잃어버리면 **주운 사람이 그대로** 씁니다 — 사번 카드와 같아요.

02 JWT = 만료되는 입장권

로그인하면 서버가 **서명한 토큰**을 줍니다. "누구·언제까지"가 토큰에 적혀 있고, 위조하면 서명이 깨져요. 서버는 **서명만 확인**하면 되니 매번 DB를 안 봐도 됩니다.

03 OAuth = 대리 신원확인

"구글로 로그인"이 대표적이죠. 우리 서버는 비밀번호를 **직접 받지 않고**, 구글이 확인한 결과만 넘겨받습니다. 신원 확인을 외부에 **위임**하는 방식이에요.

04 그래서 이 과정은 API Key

우리 API는 사람이 아니라 **프로그램**이 부릅니다. 로그인 화면도 세션도 필요 없으니, 가장 단순한 출입증 한 장 — API Key가 딱 맞습니다.

세 방식의 흐름 비교 — API Key는 키 한 장, JWT는 발급받은 입장권, OAuth는 신원 확인을 외부에 맡긴다.

토큰 인증은 어떻게 흐르나 — 로그인부터 갱신까지

API Key는 만료가 없는 출입증, 토큰은 만료 시각이 박힌 입장권입니다. 구현은 이 과정 범위 밖이라 흐름만 봅니다.

STEP 01

로그인

사용자가 아이디·비밀번호로 신원을 밝힙니다. OAuth라면 "구글로 로그인"처럼 외부 서비스가 이 단계를 대신해 줍니다.



STEP 02

토큰 발급

서버가 JWT를 만들어 줍니다. 안에 사용자 ID·만료 시각이 담기고, 서버 비밀키로 서명돼 한 글자만 고쳐도 위조가 들통납니다.



STEP 03

토큰 제시

이후 요청마다 `Authorization: Bearer <토큰>` 헤더에 싣습니다. API Key를 헤더에 싣던 것과 **자리는 같습니다** — 내용물이 다를 뿐이죠.



STEP 04

서명만 검증

서버는 DB를 뒤지지 않고 **서명만 확인**해 통과시킵니다. 토큰이 스스로를 증명하니 서버는 상태를 기억할 필요가 없습니다.

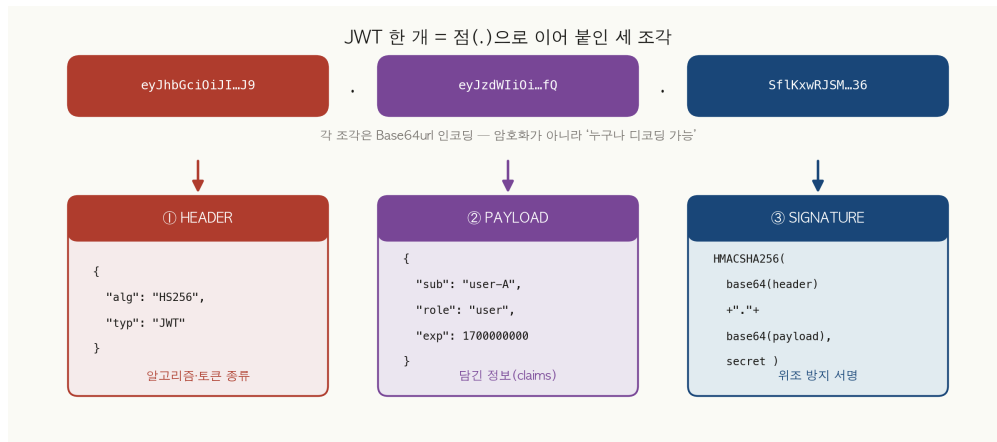


STEP 05

만료 → 갱신

토큰은 만료가 짧아(예: 15분) 유효해도 곧 무효가 됩니다. 만료되면 **리프레시 토큰**으로 새 토큰을 받아요 — 재로그인 없이. 이 두 토큰 이야기는 **다음 장**에서 자세히 봅니다.

JWT — 점 두 개로 나뉜 '입장권'의 구조



실제 JWT의 생김새 — 점(.)으로 나뉜 세 조각(header.payload.signature). 각 조각을 Base64url 디코딩하면 위 색깔별 내용이 나온다. (jwt.io 디버거의 시각화 방식)

01 생김새 = aaaa.bbbb.cccc

JWT는 점 두 개로 나뉜 **세 조각**입니다 — `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3VzZXQ6PSJlKXxwRjSM...`

각 조각은 Base64로 인코딩돼 그대로 읽히진 않지만, **암호화는 아니예요** — 누구나 디코딩할 수 있습니다.

02 ① Header — 메타데이터

토큰 **종류(JWT)**와 **서명 알고리즘**(예: HS256)이 담깁니다. "이건 이렇게 서명된 토큰"이라는 안내문이죠.

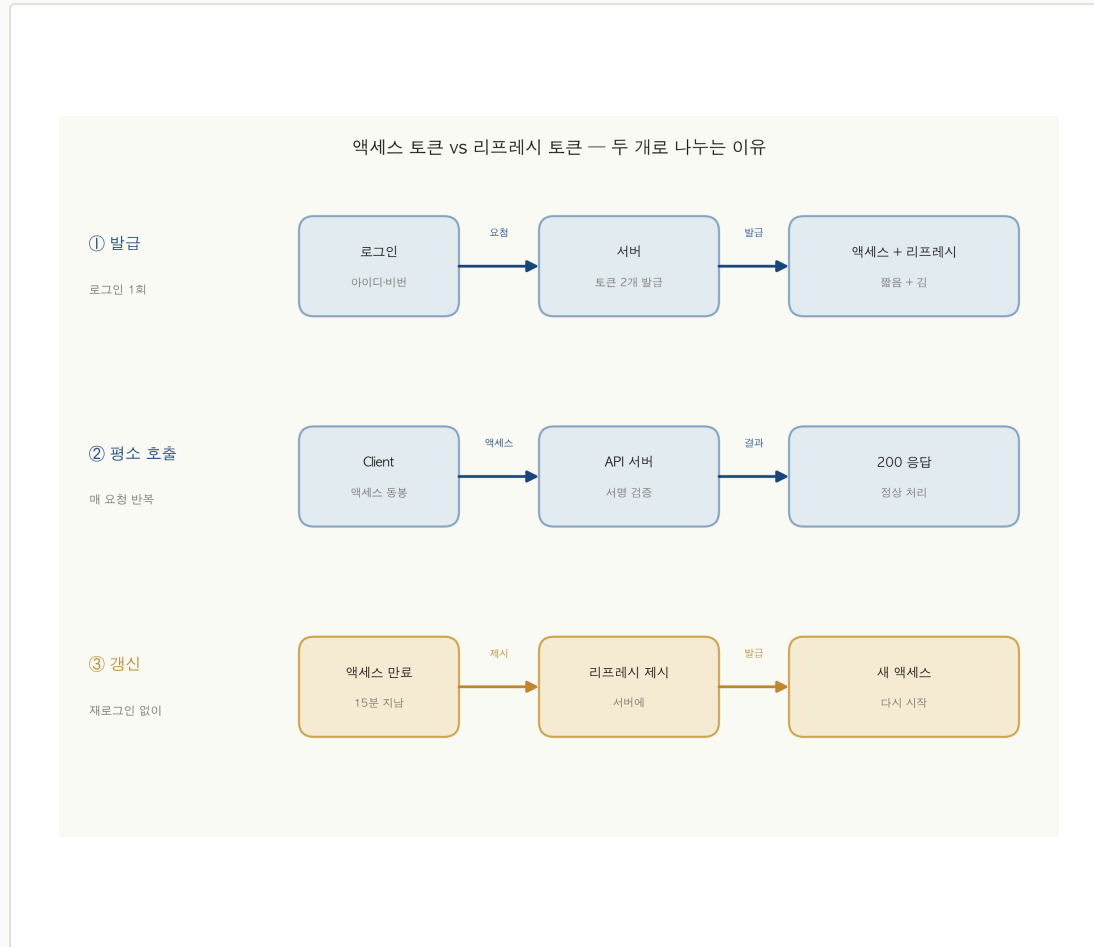
03 ② Payload — 주장(claims)

사용자 ID·권한·만료 시각(exp) 등이 담깁니다. 서버가 "넌 누구고 언제까지 유효한가"를 여기서 읽어요. 단, 누구나 볼 수 있으니 **비밀번호 같은 건 넣지 않습니다**.

04 ③ Signature — 위조 방지

Header+Payload를 **서버 비밀키로 서명**한 값입니다. 한 글자만 바뀌도 서명이 안 맞아 **위조가 들통나요** — 그래서 서버는 DB 없이 서명만 확인하면 됩니다(§1.3).

액세스 토큰 vs 리프레시 토큰 — 왜 두 개일까



01 액세스 토큰 — 짧게 산다

API 요청마다 동봉하는 토큰입니다. 수명이 **짧아(예: 15분)** 유출돼도 곧 무효가 돼요. 자주 노출되는 만큼 빨리 만료시키는 게 핵심입니다.

02 리프레시 토큰 — 길게, 안전하게

새 액세스 토큰을 받기 위한 토큰입니다. 수명이 길어(예: 2주) 안전한 곳에 보관하고, 갱신할 때만 꺼내 씁니다. 거의 노출되지 않아요.

03 갱신은 재로그인 없이

액세스 토큰이 만료되면 리프레시 토큰을 서버에 내밀어 **새 액세스 토큰**을 받습니다. 사용자는 다시 로그인할 필요가 없죠 — 끊임 없는 경험이에요.

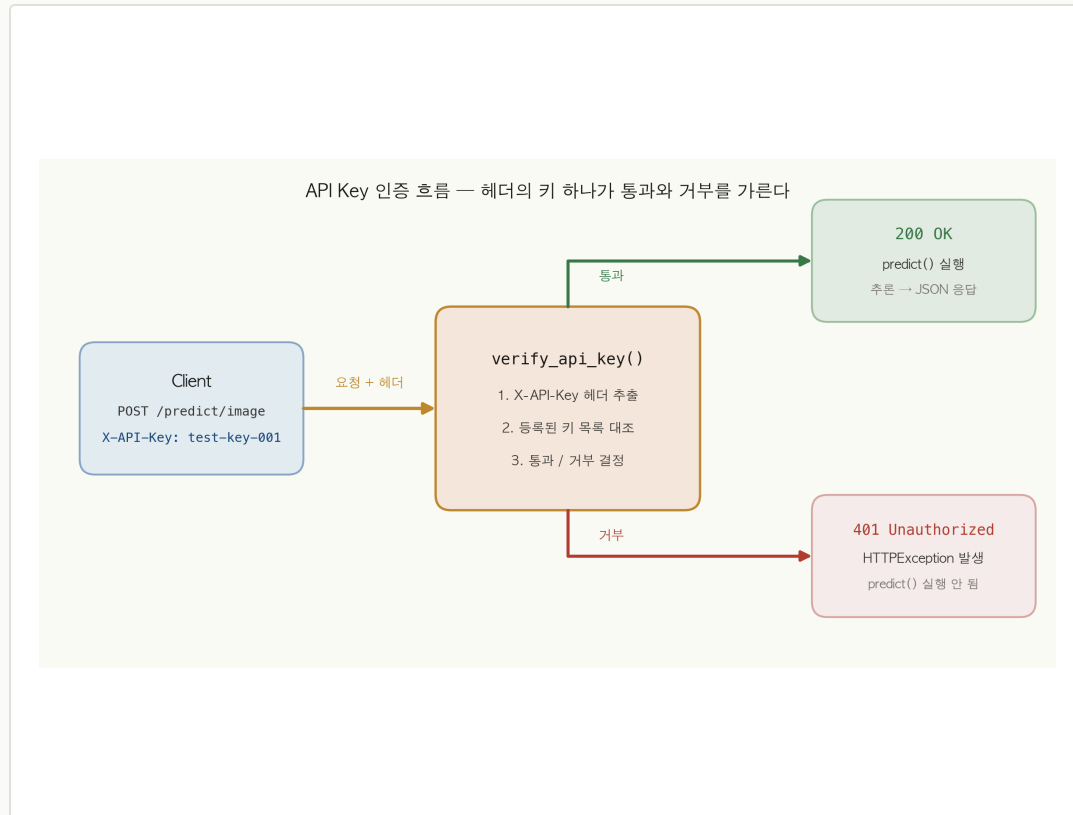
04 왜 둘로 나누나 — 보안 vs 편의

하나뿐이면 **짧게**(자주 로그인, 불편)와 **길게**(유출 위험 큼) 중 하나만 고릅니다. 둘로 나누면 짧은 노출(액세스) + 드문 사용(리프레시)으로 둘 다 잡아요.

액세스 토큰은 매 요청에 쓰고 빨리 만료, 리프레시 토큰은 새 액세스 토큰을 받을 때만 쓴다.

헤더의 키 하나가 통과와 거부를 가른다

API Key 인증의 전부입니다 — 클라이언트가 `X-API-Key` 헤더에 키를 싣고, 서버는 등록 목록과 대조합니다. 맞으면 함수 실행, 틀리면 **401**. 문지기 함수 하나가 모든 요청 앞을 지키는 그림이에요.



01 키는 헤더에 싣는다

`X-API-Key: test-key-001` 처럼 **헤더**에 담습니다. 본문은 데이터의 자리, 헤더는 부가정보의 자리 — 인증 정보와 입력 데이터가 깔끔히 분리됩니다.

02 서버는 대조만 한다

헤더에서 키를 꺼내 등록 목록에 있는지 봅니다. 있으면 "사용자A"처럼 **누구인지**까지 알아내요 — 이 이름이 §6.5 접근 로그의 재료가 됩니다.

03 통과 — 함수 실행

키가 유효하면 엔드포인트 함수가 사용자 정보와 함께 돌고, 평소처럼 200으로 답합니다. 정상 사용자는 인증을 **의식조차 못** 하죠.

04 거부 — 401, 실행 없음

키가 없거나 틀리면 **401 Unauthorized**. 중요한 건 엔드포인트 함수가 **실행조차 안 된다**는 점입니다 — 추론 비용이 한 푼도 안 들어요.

auth.py — 문지기는 함수 하나면 된다

§2.1의 그림을 코드로 옮깁니다. 헤더를 받고, 대조하고, 거르는 **함수 하나**가 인증의 전부예요. 평범한 함수라 서버 없이도 노트북에서 먼저 단독 테스트할 수 있습니다 — 올바른 키·틀린 키·키 없음 세 갈래를 미리 확인해 두세요.

```
# app/auth.py
from fastapi import HTTPException, Header

# 등록된 키 → 사용자 (학습용 하드코딩 — 실무 방식은 §2.8에서)
VALID_API_KEYS = {"test-key-001": "사용자A",
                  "test-key-002": "사용자B"}

async def verify_api_key(
    x_api_key: str = Header(None)  # X-API-Key 헤더 → 변수
) -> str:
    if x_api_key is None:          # 키 자체가 없음
        raise HTTPException(401, "API Key 필요")
    if x_api_key not in VALID_API_KEYS:  # 등록 안 된 키
        raise HTTPException(401, "유효하지 않은 키")
    return VALID_API_KEYS[x_api_key]  # 통과 → 사용자 이름
```

01 Header(None)

요청의 `X-API-Key` 헤더 값을 `x_api_key` 인자로 넣어 줍니다. 헤더가 아예 없으면 `None` 이 들어와요 — 그래서 첫 번째 `if` 가 '키 없음'을 잡습니다.

02 이름 자동 매핑

파이썬 변수 `x_api_key` 를 FastAPI가 HTTP 헤더 `X-API-Key` 로 알아서 변환합니다 — 밑줄은 하이픈으로, 대소문자는 무시. Day 2의 '선언하면 자동'이 헤더에서도 통하죠.

03 두 갈래 모두 401

키가 **없을 때도**, **틀렸을 때도** 401입니다. 메시지만 달리해 디버깅을 돕되, 상태 코드는 같아요 — 둘 다 "신원 확인 실패"라는 같은 종류의 문제니까요.

04 반환값이 곧 신원

통과하면 `"사용자A"` 를 돌려줍니다. 이 반환값이 엔드포인트의 `user` 변수로 흘러들어, "누가 불렀는지"를 응답·로그에 남기는 재료가 됩니다.

"너 누구야?"와 "넌 그거 돼?"는 다른 질문이다

사원증을 찍고 회사에 들어왔다고 서버실 문까지 열리진 않죠. **인증(authentication)**은 "누구인지" 확인하는 일, **인가(authorization)**는 확인된 사람이 "그 행동을 해도 되는지" 따지는 일입니다. 순서대로 일어나는 **별개의 관문**이에요.

01 인증이 먼저

요청이 오면 **키가 맞는지**부터 봅니다. 신원이 확인돼야 다음 질문이 가능하죠. 오늘의 `verify_api_key`가 정확히 이 첫 관문입니다.

02 인가는 그다음

신원이 확인돼도 권한은 별개입니다. "사용자A는 추론은 되지만 모델 삭제는 안 됨"처럼, **같은 사람이라도 행동마다** 허용이 갈릴 수 있어요.

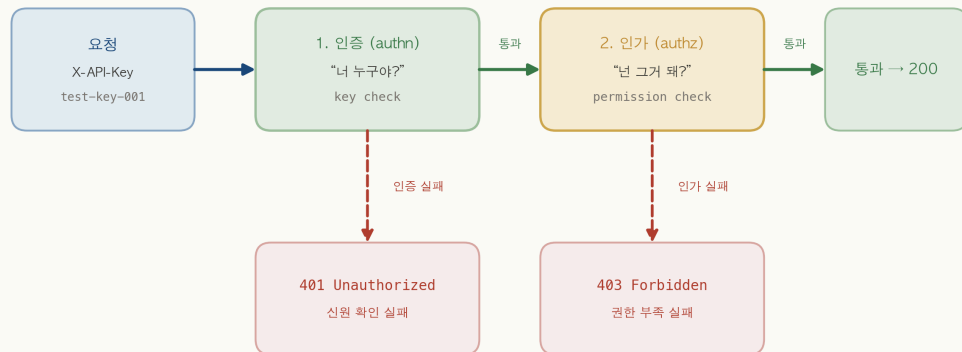
03 실패 코드가 갈린다

인증 실패는 **401**, 인가 실패는 **403**. 키가 없거나 틀리면 401, 키는 맞는데 그 리소스에 권한이 없으면 403입니다. 다음 장에서 나란히 비교해요.

04 오늘의 범위

이 과정은 **인증(401)**까지만 구현합니다. 인가는 역할·등급 설계가 따라오는 큰 주제라, "이런 관문이 하나 더 있다"는 지도만 챙겨 두면 충분해요.

인증(Authentication) vs 인가(Authorization) — 순서대로 거치는 두 관문



Depends() 한 줄이면 자물쇠가 걸린다

문지기 함수를 만들었으니 문 앞에 세울 차례입니다. 엔드포인트 인자에 `user: str = Depends(verify_api_key)` 한 줄만 더하면 끝 — 그 엔드포인트는 이제 키 없이 통과할 수 없습니다.

```
from fastapi import FastAPI, Depends
from app.auth import verify_api_key

app = FastAPI()

@app.get("/health")                # 자물쇠 없음 - 누구나 OK
async def health():
    return {"status": "healthy"}

@app.post("/predict")              # 자물쇠 있음 - 키 필요
async def predict(
    request: PredictRequest,
    user: str = Depends(verify_api_key), # ← 이 한 줄이 인증
):
    print(f"요청한 사용자: {user}")    # "사용자A"
    ...
```

01 Depends = 먼저 실행해라

"이 함수를 돌리기 전에 **먼저 이것부터** 실행해라"라는 선언입니다. 인증 말고도 DB 연결·권한 확인 등 '본 작업 전 준비물'에 두루 쓰이는 FastAPI의 핵심 기능이에요.

02 실행 순서

요청이 오면 `predict()` 보다 `verify_api_key()` 가 **먼저** 돕니다. 문지기를 통과해야 본 함수 차례 — 실패하면 본 함수는 시작조차 안 합니다.

03 성공하면 값이 흐른다

문지기의 반환값(`"사용자A"`)이 `user` 인자로 들어옵니다. 검증 **결과**를 본 함수가 바로 받아 쓰는 구조라, 전역 변수도 중복 코드도 없어요.

04 엔드포인트별 선택

`/health` 엔 자물쇠가 없습니다 — 운영 장비가 키 없이 두드리는 통로니까요. **어디에 자물쇠를 달지**를 엔드포인트 단위로 고르는 게 이 패턴의 힘입니다.

"누군지 모르겠다"와 "권한이 없다"는 다르다



401은 인증 실패, 403은 인가 실패 — 클라이언트가 할 일이 다르다.

01 401 Unauthorized

"너 누구인지 모르겠다." 키가 없거나 틀린 상태입니다. 이름은 Unauthorized지만 실제 의미는 **미인증**이에요 — 오늘의 `verify_api_key` 가 던지는 코드입니다.

02 403 Forbidden

"누군지는 알겠는데, 그건 안 돼." 유효한 키로 **권한 밖** 리소스를 부른 상태입니다. 키를 다시 보내도 소용없어요 — 권한 자체가 없으니까요.

03 클라이언트의 다음 행동

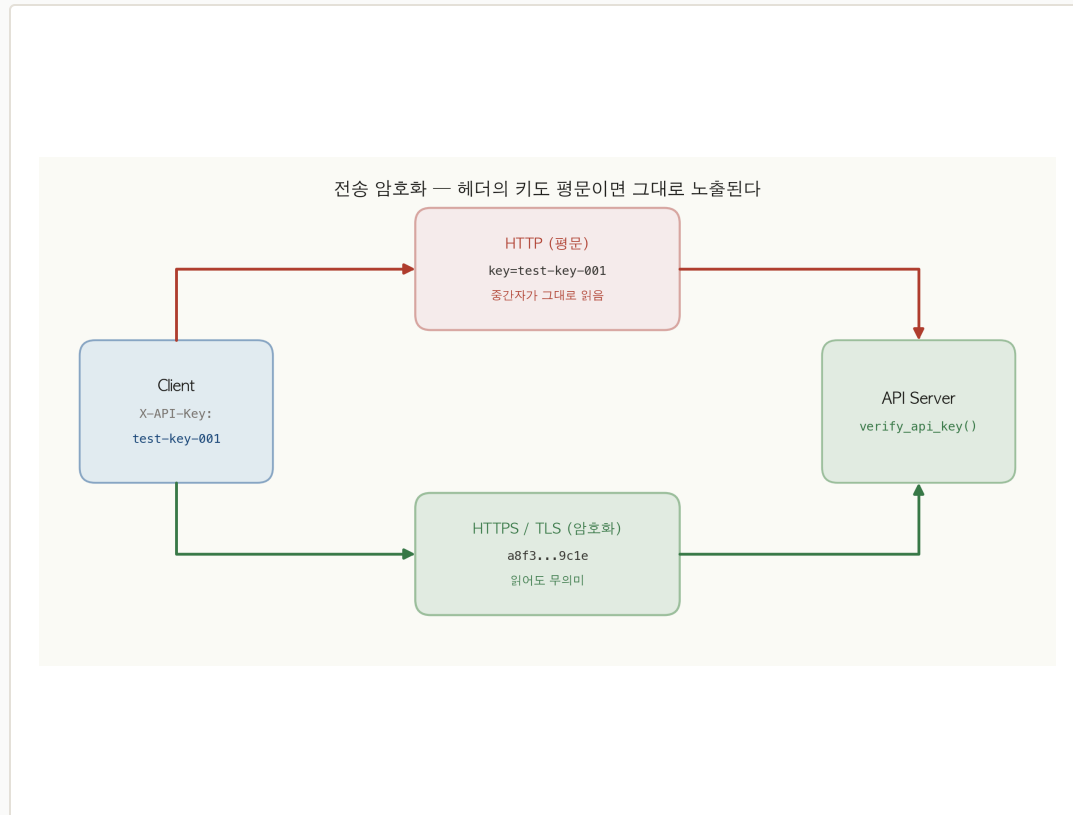
401을 받으면 **올바른 키를 구해 재시도**하면 됩니다. 403은 키 문제가 아니라 **권한 문제**라 관리자에게 요청해야 하죠. 코드 하나가 대처법을 알려 줍니다.

04 뭉뚱그리면 생기는 일

둘 다 401로 통치면, 멀쩡한 키를 가진 사용자가 "키가 틀렸나?" 하며 헛수고를 반복합니다. Day 2의 422/500 구분과 같은 원칙 — **정확한 코드가 좋은 API**입니다.

전송 중인 키 — 평문이면 길에서 그대로 샌다

인증 코드가 아무리 완벽해도, 키가 전송 중에 새면 끝입니다. API Key는 매 요청 헤더에 담겨 네트워크를 지나갑니다.
연결이 **HTTP(평문)**면 같은 네트워크의 누구나 키를 그대로 읽고, **HTTPS(TLS)**면 암호화돼 도청해도 의미가 없습니다.



01 헤더 ≠ 암호화

흔한 오해입니다 — 헤더에 넣었다고 숨겨지는 게 아닙니다. 평문 통신에선

`X-API-Key: test-key-001` 이 카페 와이파이의 도청에 **글자 그대로** 찍혀요.

02 TLS가 길을 암호화

HTTPS = HTTP + **TLS**. 클라이언트-서버 사이 전 구간이 암호화돼, 도청자는 **해독 불가능한 바이트**만 봅니다. 키도 데이터도 함께 보호되죠.

03 인증의 전제 조건

키 검증 로직이 완벽해도 전송이 평문이면 키가 새고, 새는 순간 공격자가 '정상 사용자'로 둔갑합니다. **인증은 HTTPS 위에서만** 의미가 있어요.

04 실무에선 어디서 하나

보통 FastAPI 앞단의 프록시·로드밸런서가 **암호화 처리**를 맡습니다. 우리 코드는 그대로여도, 배포 때 "https로 노출됐는지"는 반드시 확인할 체크리스트입니다.

TLS는 어떻게 암호화 채널을 여나 — 악수 한 번



TLS 1.3 — 왕복 한 번에 암호 합의가 끝나고, 그 뒤 모든 데이터가 암호화된다.

01 ClientHello

클라이언트가 "나는 이런 암호들을 쓸 줄 알아"라며 지원 목록과 **임시 키 재료**를 먼저 내립니다. 본 데이터가 오가기 전의 첫인사예요.

02 ServerHello + 인증서

서버가 암호를 하나 고르고, **인증서**로 "내가 진짜 그 서버"임을 증명하며 키 재료를 돌려줍니다. 가짜 서버에 키를 보내는 일을 인증서 검증이 막아 주죠.

03 왕복 한 번이면 끝

현재 표준 **TLS 1.3**은 이 합의를 왕복 **한 번**에 끝냅니다. 첫 요청 지연이 줄고, 훗날 서버 키가 새도 과거 통신은 안전하게 남는 설계예요.

04 그 위로 키가 흐른다

악수가 끝난 뒤에야 **X-API-Key**가 암호화된 길 위를 지나갑니다. §2.6의 결론과 같은 말 — **TLS가 먼저 깔리고, 인증은 그 위에서 이뤄집니다.**

키를 코드에 적는 순간, Git이 영원히 기억한다

오늘의 하드코딩은 학습용입니다. 실무에서 비밀은 **코드 밖**에 둡니다 — 코드엔 이름만, 값은 환경에.

나쁜 예 — 하드코딩

코드에 박힌 키

커밋 한 번이면 끝

키를 코드에 적고 push하면 GitHub·CI 로그·동료의 클론까지 키가 **복제**됩니다. 실제로 공개 저장소에서 키를 굵는 붓이 분 단위로 돌고 있어요.

지워도 남는다

Git은 과거를 버리지 않습니다. 다음 커밋에서 지워도 **히스토리에서 복구** 가능해요. 한 번 올라간 키는 '유출됐다'고 치고 통째로 폐기·재발급해야 합니다.

교체도 배포다

키를 바꾸려면 코드를 고치고 다시 배포해야 합니다. 유출 대응이 **코드 릴리스 속도**에 묶이는 셈이죠 — 비상시에 가장 느린 길입니다.

좋은 예 — 코드 밖으로

환경변수 · 시크릿 금고

.env + 환경변수

키 **값**은 `.env` 파일에만 두고, 코드는 `os.environ["API_KEY"]` 로 **이름만** 읽습니다. `.env` 를 `.gitignore` 에 넣으면 저장소엔 키의 흔적이 없어요 — 형식 공유는 키 없는 `.env.example` 로.

시크릿 금고

팀·운영 규모에선 전용 비밀 관리 서비스(클라우드 시크릿 매니저 등)를 씁니다. 접근 권한·감사 기록·자동 교체까지 금고가 관리해 주죠.

교체가 쉬워진다

키가 코드 밖에 있으면 유출 시 **값만 갈아끼우고 재시작**하면 끝입니다. 코드 수정도 재배포도 없이 — 주기적 키 교체가 비로소 현실적인 운영이 됩니다.

인증만으로는 부족하다 — 세 가지 방어선 더

인증은 '누가'를 거릅니다. 하지만 §1의 서버 다운·비용 폭탄은 **허용된 사용자**도 일으킬 수 있습니다. 인증 위에 엮는 방어선 셋을 짚습니다. 구현은 범위 밖이라 개념만 봅니다.

사용량 제한(rate limiting). "한 키는 분당 60회까지"처럼 호출 횟수에 상한을 둡니다. 무차별 대입과 비용 폭탄을 직접 막는 장치입니다. 초과하면 **429 Too Many Requests**로 거절하고, 잠시 뒤 다시 받습니다.

입력 크기·시간 제한. 요청 본문 크기와 처리 시간에 상한을 둡니다. §4의 5MB 제한이 그 예이고, 추론이 너무 오래 끌면 타임아웃으로 끊습니다. 한 요청이 서버 자원을 독점하는 일을 막습니다.

CORS — 브라우저 출처 제한. 어떤 웹페이지에서 이 API를 부를 수 있는지 **출처(origin)**로 통제합니다. Day 7에서 프론트가 백엔드를 부를 때 필요해집니다. 서버 간 호출에는 영향이 없습니다.

셋 다 인증을 대신하지 않습니다. 사용량 제한·크기 제한·CORS는 인증 **위에** 쌓는 층입니다. 누구인지부터 가린 다음, 얼마나·무엇을·어디서 부르는지를 차례로 제한하는 구조입니다.

키 하나에 따라붙는 일들 — 발급부터 폐기까지

오늘 코드로 만든 건 **검증** 한 단계뿐입니다. 운영에선 키 하나에 다섯 단계가 따라붙어요 — "키를 헤더에 싣고 대조한다"로 끝나지 않습니다.

STEP 01

발급

사용자·앱마다 **고유한 키**를 만들어 줍니다. 누구 것인지 구분돼야 사용량도 권한도 키 단위로 관리돼요. 실무에선 콘솔에서 키를 만들고 **한 번만** 보여 줍니다.



STEP 02

안전 저장

키 **원문은 저장하지 않습니다**. 서버엔 **해시**만 두고 대조하고(비밀번호와 같은 원리), 클라이언트는 `.env` 시크릿 금고에 둡니다(§2.8). 코드·로그엔 남기지 않아요.



STEP 03

검증

매 요청 헤더의 키를 등록 목록과 대조합니다 — **오늘 만든** `verify_api_key`가 바로 이 단계예요. 통과는 200, 없거나 틀리면 401.



STEP 04

사용량 추적·제한

키별로 **누가 얼마나** 불렀는지 기록하고, rate limit·quota로 상한을 둡니다(§2.9). 요금 정산과 **비정상 요청 탐지**의 근거가 돼요.



STEP 05

회전·폐기

키를 **주기적으로 교체(rotation)**하고, 유출되면 즉시 **폐기(revoke)** 후 재발급합니다. 키를 코드 밖에 둔 덕에 값만 갈아끼우면 끝이에요.

숫자 8개에서, 수 MB 이미지로

Day 5의 입력은 숫자 8개짜리 JSON — 몇백 바이트에 형식도 일정했습니다. 오늘 받을 이미지는 **크기도 형식도 제멋대로**입니다. 입력의 성격이 바뀌면 API의 일도 바뀝니다.

정형 데이터는 작고 일정합니다. 필드 이름과 타입이 스키마로 고정돼 있죠.
Pydantic 검증을 통과하면 **정규화 → 텐서 변환**으로 처리가 끝났습니다.

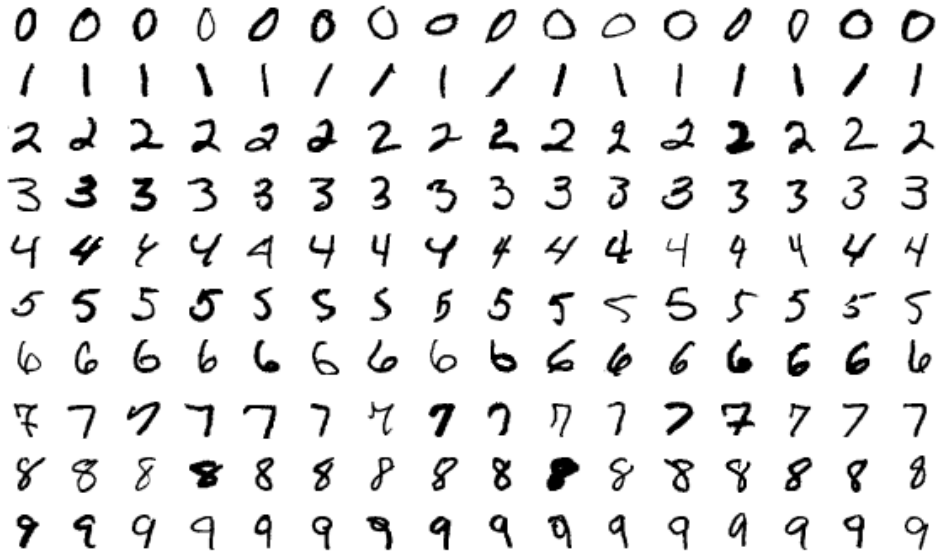
비정형 데이터는 크고 가변적입니다. PNG·JPEG 이미지는 수 KB에서 수 MB까지 들쭉날쭉하고, 해상도·색상 모드도 제각각입니다. "필드가 있고 타입이 맞는지"만 봐서는 아무것도 보장되지 않아요.

그래서 단계가 늘어납니다. **파일 검증 → 디코딩 → 리사이징 → 정규화 → 텐서 변환**. 정형 데이터엔 없던 앞쪽 세 단계가 새로 붙습니다 — 파일은 열어 보기 전까지 내용물을 알 수 없으니까요.

검증이 핵심입니다. 진짜 이미지인지, 크기가 적절한지, 깨지지 않았는지 — 이걸 **모델에 닿기 전에** 확인해야 합니다. §1의 인증이 '누가'를 걸렀다면, 이 검증은 '무엇이' 들어오는지 거릅니다. 둘 다 보안이에요.

오늘의 목표는 이미지입니다. MNIST 손글씨 이미지를 받아 분류하는 API를 만듭니다. Day 1에서 학습해 둔 모델을 그대로 쓰니, 새로 배우는 건 **받고 검사하는 법**뿐입니다.

오늘 분류할 것 — MNIST 손글씨 숫자



MNIST 손글씨 숫자 예시 — 28×28 흑백 이미지. 오늘 API가 업로드받아 분류할 바로 그 데이터다. 출처: MnistExamples, Wikimedia Commons(CC BY-SA).

01 28×28 흑백 한 장

MNIST는 0~9 손글씨를 **28×28 픽셀 흑백**으로 담은 고전 데이터셋입니다. 한 장이 곧 784개 숫자 — Day 1에서 학습해 둔 모델이 이걸 받아 숫자를 맞혀요.

02 정형과 다른 점

Day 5의 입력은 숫자 8개였죠. 이건 한 장이 **수백 바이트~수 MB의 이미지 파일**입니다. 그래서 §3.2부터 '파일을 어떻게 받나'가 새 과제가 돼요.

03 전처리가 필요하다

업로드된 PNG/JPEG를 **디코딩 → 28×28 리사이즈 → 정규화**해야 모델 입력 형태가 됩니다 — §4·§5의 검증·파이프라인이 바로 그 일이에요.

04 왜 MNIST인가

작고 단순해 **배포의 흐름**에 집중할 수 있습니다. 진짜 서비스엔 더 큰 이미지가 오지만, 받고-검사하고-추론하는 **빠대는 똑같습니다**.

Base64로 담을까, 파일로 보낼까

이미지 전송 두 방식 — JSON 안에 담기 vs 파일 그대로 보내기

Base64 in JSON

Content-Type: application/json

```
{ "image": "iVBORw0KGgoAAA...NSUhEUg==", "meta": ... }
```

- + 다른 JSON 필드와 한 요청에
- 크기 약 +33% 증가
- 인코딩/디코딩 단계 필요

Day 2~4 에서 사용

UploadFile (multipart)

Content-Type: multipart/form-data

```
file: digit.png  
(원본 바이트 그대로)
```

- + 인코딩 없이 원본 크기
- + Swagger 파일 선택 버튼 자동
- 다른 JSON과 한 요청은 번거로움

Day 6 에서 사용

01 Base64 — JSON에 담기

이미지 바이트를 문자열로 바꿔 JSON 필드에 넣습니다. Day 4에서 쓴 방식으로, **다른 JSON 필드와 한 요청에** 묶어야 할 때 유리해요. 대신 크기가 약 +33% 큼니다.

02 UploadFile — 그대로 보내기

파일을 폼 업로드 방식(multipart)으로 직접 보냅니다. 인코딩이 없어 원본 크기 그대로고, 받는 쪽도 디코딩 단계가 빠집니다. HTML 폼의 파일 업로드와 같은 표준 방식이에요.

03 받는 코드는 선언 하나

엔드포인트 인자에 `file: UploadFile` 만 적으면 받기 끝입니다.

`file.filename` (원본 이름)과 `file.content_type` (신고된 형식)이 따라오는데 — 둘 다 **클라이언트가 적어 보내는 값**이라 그대로 믿으면 안 돼요(§4).

04 오늘의 선택

UploadFile입니다. 이미지 한 장만 받으면 되는 데다, 문서 화면(/docs)에 **파일 선택 버튼**이 자동으로 생겨 테스트가 클릭 몇 번으로 끝나거든요 — §6.3에서 직접 확인합니다.

JSON 안에 문자열로 담는 Base64 vs 파일을 그대로 보내는 UploadFile.

인증된 사용자도 잘못된 파일을 보낸다

인증(§2)으로 '누가'는 걸렸습니다. 하지만 올바른 키를 가진 사용자가 보낸 파일이 멀쩡하다는 보장은 없습니다. 악의가 아니라 실수로도 위험한 입력은 들어옵니다.

100MB 이미지가 오면 서버 메모리가 차오릅니다. 한 요청이 메모리를 독점하면 같은 서버의 **다른 사용자까지** 느려지거나 죽어요. 크기 상한 없는 업로드는 서버를 멈추게 하는 가장 쉬운 통로입니다.

실행 파일을 .png 로 위장해 보내면 이미지로 열다 에러가 나거나, 최악엔 보안 사고로 번집니다. 확장자도 신고된 형식도 클라이언트가 **마음대로 적는 값**이라, 신고만 믿어선 안 됩니다.

10000×10000 픽셀 이미지는 디코딩·전처리에 CPU와 시간을 통째로 잡아먹습니다. 파일 크기는 작아도(압축 덕에) 펼치면 거대한, 이른바 **압축 폭탄**류의 입력이죠.

손상된 파일은 이미지로 여는 순간 예외를 터뜨립니다. 잡지 않으면 500 — 클라이언트는 '서버 잘못'으로 오해하고, 우리는 멀쩡한 코드를 의심하며 시간을 버립니다.

그래서 모델 앞에 검문소를 둡니다. 이 입력들을 **모델에 닿기 전에** 막고, 막을 땐 400으로 "네 파일이 문제"라고 정확히 알려줍니다. Day 2에서 Pydantic이 JSON에 해 준 일을, 파일에 대해선 우리가 직접 짜는 거예요.

이미지를 여는 손 — Pillow(PIL)



Pillow — 파이썬의 표준 이미지 처리 라이브러리. 업로드된 바이트를 **진짜 이미지로 열고**, 크기·모드를 바꾼다.
출처: Python Pillow 프로젝트 로고(HPND/오픈소스).

01 디코딩 — 열어 봐야 안다

`Image.open(BytesIO(데이터))` 한 줄로 업로드 바이트를 이미지 객체로 엽니다.
깨진 파일이면 여기서 예외가 나니, 이 시도 자체가 1차 검증이 돼요.

02 리사이즈 — 모델 입력 크기로

`img.resize((28, 28))` 로 어떤 해상도가 와도 모델이 기대하는 **28×28**로 맞추습니다. 색상 모드도 `convert("L")` 로 흑백 통일하죠.

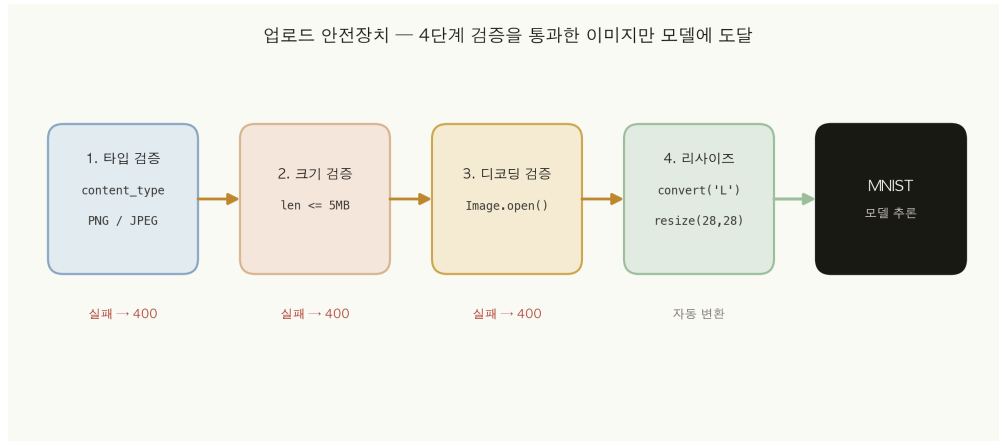
03 .verify() — 진짜 이미지인가

확장자만 `.png` 라고 진짜 이미지는 아닙니다. Pillow로 **실제로 열어 봐야** 위장한 파일을 걸러내요 — §4.2 검문의 핵심 도구입니다.

04 왜 직접 안 짜나

이미지 포맷(PNG·JPEG·...)을 손수 파싱하는 건 위험하고 번거롭습니다. **검증된 라이브러리**에 맡기는 게 안전 — 보안에서 '바퀴를 다시 발명하지 않기'의 한 예죠.

타입 → 크기 → 디코딩 → 리사이즈, 순서대로 거른다



01 1. 타입 검증

신고된 형식이 PNG-JPEG인지 봅니다. 실행 파일을 `.png`로 위장해도 형식이 다르다면 여기서 **400**. 가장 싼 검사를 **가장 먼저** 뒤서, 명백한 위반은 읽기도 전에 끊어요.

02 2. 크기 검증

내용을 읽어 길이를 잡니다 — 5MB 초과면 **400**. §4.1의 '100MB 폭탄'이 메모리를 점령하기 전에 끊는 방어선입니다.

03 3. 디코딩 검증

이미지를 **실제로 열어 봅니다**. 신고가 image/png여도 내용물이 쓰레기일 수 있어요 — 신고값이 아니라 **실물**로 확인하는 단계입니다. 실패하면 역시 400.

04 4. 리사이즈 + 흑백 변환

이 단계만은 거절이 아니라 **변환**입니다 — 어떤 해상도가 와도 모델 입력 규격(28×28 흑백)으로 맞춰요. 통과한 깨끗한 이미지만 모델에 도달합니다.

1~3단계는 실패 시 400으로 거절, 4단계는 어떤 이미지든 모델 입력 규격으로 변환한다.

image_utils.py — 검문소를 함수 하나로

4단계 검문을 함수 하나에 **순서대로** 담습니다. 통과하면 28×28 흑백 이미지를 돌려주고, 어느 단계든 걸리면 그 자리에서 400을 던져요. 이 함수도 서버 없이 노트북에서 먼저 단독 테스트해 두세요 — 통합 때 검증 로직을 의심 목록에서 지울 수 있습니다.

```
# app/image_utils.py
from fastapi import UploadFile, HTTPException
from PIL import Image
import io

ALLOWED = {"image/png", "image/jpeg", "image/jpg"}
MAX_SIZE = 5 * 1024 * 1024 # 5MB

async def validate_and_read_image(file, max_size=MAX_SIZE,
                                   target_size=(28, 28)):
    if file.content_type not in ALLOWED: # 1. 타입
        raise HTTPException(400, "지원하지 않는 형식")
    contents = await file.read() # 2. 크기
    if len(contents) > max_size:
        raise HTTPException(400, "파일이 너무 큼니다")
    try: # 3. 디코딩
        image = Image.open(io.BytesIO(contents))
    except Exception:
        raise HTTPException(400, "읽을 수 없는 이미지")
    return image.convert("L").resize(target_size) # 4. 변환
```

01 싼 검사부터

타입(문자열 비교) → 크기(길이) → 디코딩(실제 파싱) 순으로 **비용이 싼 검사가 앞**입니다. 잘못된 입력일수록 일찍, 싸게 떨어져 나가는 배치예요.

02 read 후 크기 체크

`await file.read()` 로 전체를 읽고 길이를 켓니다. 이 시점에 메모리에 올라오므로, 진짜 대용량 서비스라면 **조금씩 읽으며** 끊는 개선도 가능합니다 — 원리는 같아요.

03 except는 넓게, 응답은 좁게

이미지 열기 실패의 원인은 다양하지만, 클라이언트에겐 "읽을 수 없는 이미지" 한 가지로 답합니다. 내부 사정은 기록에, 바깥엔 **행동 가능한 메시지**만.

04 반환값이 곧 계약

이 함수가 돌려주는 건 언제나 **28×28 흑백**입니다. 호출하는 쪽(§6.1)은 검증을 전혀 신경 쓰지 않죠 — 관심사 분리가 여기서도 반복됩니다.

업로드 한 번에, 여섯 단계가 흐른다



인증 → 파일 검증 → 전처리 → 추론 → 후처리 → JSON 응답. 새것은 앞의 두 관문뿐이다.

01 두 관문이 먼저

인증(§2)이 "누구냐"를, **파일 검증**(§4)이 "무엇이냐"를 묻습니다. 어느 쪽이든 걸리면 모델 근처에도 못 가요 — 비싼 연산은 검증된 요청에만 씁니다.

02 전처리

검증을 통과한 28×28 이미지를 학습 때와 **같은 정규화**로 텐서화합니다. 상수가 어긋나면 에러 없이 조용히 오답만 냅니다 — Day 5의 약속 그대로예요.

03 추론은 전용 풀로

무거운 추론을 별도 일꾼(스레드)에 맡깁니다. Day 3에서 배운 그 패턴 — 계산이 서버를 막아 다른 요청까지 세우는 일을 피합니다.

04 후처리 → 응답

예측 클래스와 확신도를 뽑아 `{"label": "7", "confidence": 0.98}` 로 답합니다. 응답에 **user**도 실어 "누가 불렀는지"가 끝까지 따라가게 해요.

image_api.py — 인증 + 검증 + 추론을 한 함수로

이제 전부 합칩니다. 인증은 `Depends` 한 줄, 검증은 함수 호출 한 줄 — §2와 §4를 미리 모듈로 빼 둔 덕에, 엔드포인트 본문은 파이프라인 순서 그대로 읽힙니다.

```
@app.post("/predict/image", tags=["Inference"])
async def predict_image(
    file: UploadFile = File(..., description="분류할 이미지"),
    user: str = Depends(verify_api_key),  # 관문 1 — 인증
):
    logger.info(f"추론 요청 — 사용자: {user}")  # 접근 로그
    # 관문 2 — 파일 검증 + 28x28 그레이스케일 로드
    image = await validate_and_read_image(file, target_size=(28, 28))
    # 전처리 — 텐서 변환 → (1, 1, 28, 28)
    tensor = img_transform(image).unsqueeze(0)
    # 추론 — 무거운 계산은 전용 풀로 (Day 3 패턴)
    loop = asyncio.get_event_loop()
    result = await loop.run_in_executor(executor, run_inference, tensor)
    # 응답 — 누가 불렀는지(user)도 함께
    return {"success": True, "user": user,
            "label": result["predicted_class"],
            "confidence": round(result["confidence"], 4)}
```

01 관문은 한 줄씩

인자의 `Depends(verify_api_key)` 가 인증을, 본문 첫 줄의 `validate_and_read_image` 가 파일 검증을 맡습니다. 오늘 새로 생긴 파일은 `auth:image_utils:image_api` 셋뿐 — Day 1~5 파일은 그대로 두고 **조립만** 해요.

02 unsqueeze(0)

전처리가 만든 `(1, 28, 28)` 텐서 앞에 묶음(배치) 축을 붙여 `(1, 1, 28, 28)` 로 만듭니다. 모델은 언제나 '여러 장 묶음'을 기대하니, 한 장도 **1장짜리 묶음**으로 포장하는 거죠.

03 run_in_executor

추론을 전용 풀로 보냅니다. `await` 덕에 추론이 도는 동안에도 서버는 다른 요청을 받아요 — Day 3의 동시성 설계가 여기서 실전 투입됩니다.

04 로그와 응답에 user

함수 첫 줄에서 "누가"를 기록으로 남기고, 응답에도 담습니다. 응답 **계약**의 키는 `label`, 내부 조회는 `predicted_class` — 계약과 내부 키를 분리해 둡니다.

키는 헤더에, 파일은 multipart로 — 호출은 한 번

서버를 만들었으니 불러 봅니다. 호출 쪽이 챙길 건 둘뿐입니다 — **인증 키는 헤더에, 이미지는 파일(multipart)로**. 터미널의 curl과 파이썬의 httpx, 둘 다 같은 두 가지를 씁니다.

```
# 1) 터미널에서 - curl
curl -X POST http://localhost:8000/predict/image \
  -H "X-API-Key: test-key-001" \
  -F "file=@digit.png"

# 2) 파이썬에서 - httpx
import httpx

with open("digit.png", "rb") as f:
    r = httpx.post(
        "http://localhost:8000/predict/image",
        headers={"X-API-Key": "test-key-001"},          # 인증 - 헤더
        files={"file": ("digit.png", f, "image/png")},  # 파일 - multipart
    )
print(r.status_code, r.json())  # 200 {"label": "7", ...}
```

01 키는 헤더 한 줄

`-H "X-API-Key: ..."` (curl)와 `headers={...}` (httpx)가 같은 일을 합니다. 서버의 `verify_api_key` 가 이 헤더를 읽어오. 빼면 401이 돌아옵니다.

02 파일은 multipart

curl의 `-F`, httpx의 `files=` 가 §3.2의 **UploadFile** 방식입니다. (이름, 파일객체, 형식) 튜플로 보내면 서버의 `file.content_type` 이 채워집니다.

03 응답은 JSON

성공하면 `{"label": ..., "confidence": ...}` 가 옵니다. `r.status_code` 로 200·401·400을 구분하고, `r.json()` 으로 결과를 읽습니다.

04 Day 7로 이어진다

이 httpx 호출 패턴이 다음 프로젝트의 **프론트 → 백엔드** 호출에 그대로 씁니다. 사람이 아니라 프로그램이 키를 싣는다는 §1.2의 말이 여기서 코드로 드러나요.

브라우저에서 키를 넣고, 파일을 골라, 실행

코드 한 줄 없이도 시험할 수 있습니다. FastAPI가 자동 생성한

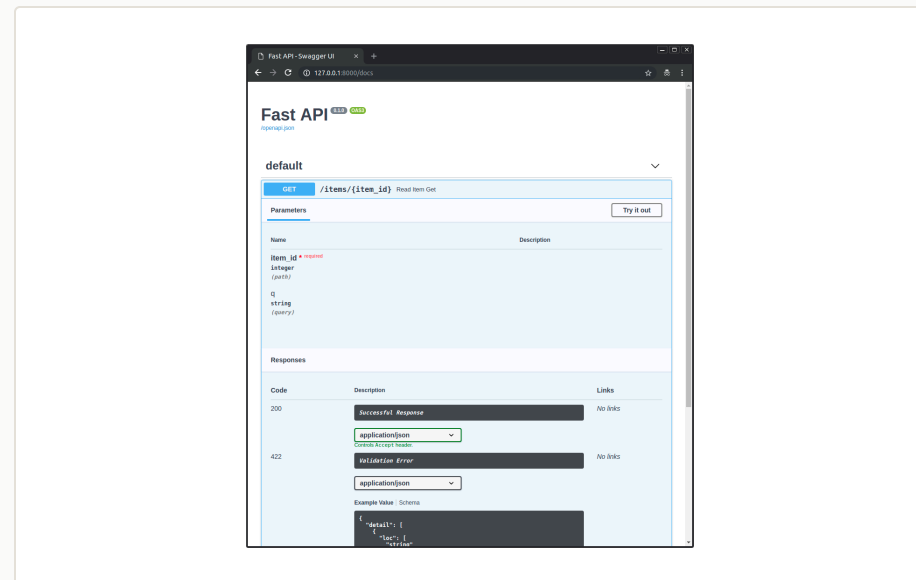
`/docs`에 인증 입력란과 파일 선택 버튼이 이미 만들어져 있거든요.

`http://localhost:8000/docs`에 접속해 `POST /predict/image`를 펼치고 **Try it out**을 누릅니다. 오늘 만든 엔드포인트가 문서로 떠 있어요.

`Header()`로 선언한 `x_api_key`가 **입력란**으로 나타납니다. `test-key-001`을 넣으면 — 우리가 짰 인증이 브라우저 UI를 공짜로 얻은 셈이죠.

`UploadFile` 덕에 **Choose File** 버튼도 자동입니다. 손글씨 숫자 이미지를 고르고 **Execute**를 누르면 `{"label": ..., "confidence": ...}`가 바로 보입니다.

키를 빼거나 `.txt`를 올려 보세요 — 401과 400이 그 자리에서 확인됩니다. 이 화면은 **제출 필수 항목**이기도 해요 — 수행 캡처와 체크포인트 답변을 MD로 정리해 GitHub 링크로 제출합니다.



실제 FastAPI `/docs` (Swagger UI) 화면 — 엔드포인트마다 **Try it out** 파라미터 응답이 자동 생성된다. 여러분의 `/predict/image`도 이렇게 떠서, 인증 입력란·파일 버튼이 그대로 나타난다. 출처: FastAPI 공식 문서(tiangolo, MIT).

"자물쇠와 검문소가, 문서와 테스트 UI까지 공짜로 데려온다."

401 · 401 · 200 · 400 — 네 갈래를 다 찢러본다

되는 것만 확인하면 절반짜리 테스트입니다. **일부러 잘못 보내서** 거절이 설계대로 작동하는지까지 봐야 해요.

STEP 01

키 없이 → 401

헤더 없이 보내면 **401**. 응답이 빨리 오는 것도 확인 포인트 — 문 앞에서 거절됐으니 **추론 비용이 전혀 들지 않았**다는 뜻이거든요.



STEP 02

틀린 키 → 401

`wrong-key` 를 실어도 역시 **401**. 없는 키와 틀린 키가 같은 코드로 답하는지 확인합니다 — 둘 다 '신원 확인 실패'니까요.



STEP 03

올바른 키 → 200

올바른 키 + 진짜 MNIST 이미지면 200과 예측이 옵니다. 여러 장을 연달아 보내 **연속 호출**도 안정적인지 보면 더 좋아요.



STEP 04

잘못된 파일 → 400

키는 맞지만 `.txt` 라 **400**. 인증 (401)과 검증(400)이 **서로 다른 코드로** 답하는 걸 확인하세요 — 클라이언트가 '키 문제'인지 '파일 문제'인지 즉시 압니다.



STEP 05

코드가 곧 문진표

401이면 키를, 400이면 파일을, 500이면 서버 기록을 봅니다. **코드만 보고 원인을 좁히는** API가 완성됐어요.

응답에 담은 user — 접근 추적의 시작점

§1에서 본 네 번째 문제, "누가 보냈는지 모른다"를 인증이 풀어 줍니다. 이제 모든 요청을 **사용자 이름과 함께** 기록할 수 있습니다.

인증이 로그에 의미를 더합니다. 인증 전 기록은 "요청이 왔다"가 전부였습니다.

`verify_api_key`가 돌려준 이름을 함께 적으면, **누가** 무엇을 추론했는지가 모든 줄에 남습니다. 접근 기록이 책임 추적의 도구가 됩니다.

새 도구는 필요 없습니다. Day 3의 로거·미들웨어가 이미 메서드·경로·상태 코드·처리 시간을 자동 기록합니다. 여기에 사용자 이름 한 칸만 더하면 "누가·언제·무엇을·결과는"이 한 줄에 모입니다.

상태 코드 분포가 경보기입니다. 401이 갑자기 폭증하면 누군가 키를 무차별 대입 중이라는 신호입니다. 한 키의 요청량이 비정상으로 치솟으면 과다 사용입니다. 코드별 집계가 공격을 숫자로 드러냅니다.

로그가 새 유출구가 되면 안 됩니다. API Key 전문이나 업로드 원본은 기록하지 마세요. 로그 파일도 유출될 수 있습니다. 키는 앞 몇 자만 남기거나 사용자 이름으로 대체합니다.

기록은 사후 분석이 아니라 사전 장치입니다. 에러율·응답시간·요청량을 모아 지켜보면, 장애를 사용자 항의보다 먼저 알아챕니다. 인증·검증·기록이 갖춰진 지금, API는 비로소 운영할 수 있는 상태가 됩니다.

오늘을 세 숫자로 기억하기

인증의 401, 검증의 400, 모델 입력의 28 — 이 셋이 오늘의 골격입니다.

401

인증 실패 — 누구나

키가 없거나 틀리면 **401 Unauthorized**.

`verify_api_key` 가 던지고, 엔드포인트는 실행조차 안 됩니다. 권한 부족의 **403**과 구분하는 것까지가 오늘의 공부예요.

400

검증 실패 — 무엇이냐

타입·크기·디코딩 검문에 걸리면 **400 Bad Request**. 인증과 다른 코드라, 클라이언트가 '키 문제'와 '파일 문제'를 즉시 구분합니다.

28

모델 입력 규격

어떤 해상도가 와도 **28×28 흑백**으로 변환해 모델에 넘깁니다. 학습 때와 같은 규격·같은 정규화 — 어긋나면 조용히 틀립니다.

자물쇠를 채우고, 들어오는 것을 검사했다

오늘 한 일

- ▶ 열린 API의 위험 4장면 — 다운·비용·도용·추적 불가
- ▶ `Header` + `Depends` 로 API Key 인증, 실패는 401
- ▶ 인증 vs 인가(401/403) · TLS · .env 시크릿 — 보안 지도
- ▶ `UploadFile` + 4단계 안전장치(타입·크기·디코딩·리사이즈, 실패 400)
- ▶ 인증+검증+추론을 한 엔드포인트로, 접근 로그에 user 기록

열린 API는 공짜 GPU다. 누구인지 확인하고(인증·401), 들어오는 것은 검사하고(검증·400), 길은 암호화(TLS), 비밀은 코드 밖(.env), 모든 출입은 기록(로그) — 이 다섯이 갖춰져야 모델이 세상에 나갈 수 있다.

다음 — DAY 7 (프로젝트 2)

- ▶ 오늘의 `auth.py`를 한 줄도 안 고치고 재사용
- ▶ LLM 챗봇 — 언어 모델을 직접 서빙한다
- ▶ 프론트가 헤더에 키를 실어 백엔드 호출
- ▶ 대화 기록을 쌓는 멀티턴 화면까지
- ▶ 백엔드-프론트를 잇는 두 번째 완성형 서비스